MFE FORMWORK MR11 SYSTEM

Code Guide: Configuration, Build and Scripts

Version 1.0

Prepared for

MFE Formwork

Prepared by

MyIntellX project team

Date

21 October 2026

Status

Draft for client review

Document control

Item	Detail
Document	Code Guide: Configuration, Build and Scripts
System	MFE Formwork MR11 System, version 1.0
Document version	1.0
Date	21 October 2026
Status	Draft for client review
Based on	Repository izzatmyintellx05/MFE, branch main, commit 0cc8c36
Prepared for	MFE Formwork
Prepared by	MyIntellX project team
Intended readers	Developers taking over the code, client IT
Classification	Confidential, for client and vendor use only

Revision history

Version	Date	Description	Author
1.0	21 October 2026	First issue for client review	MyIntellX project team

Contents

1. Introduction 4
2. Project settings 4
2.1 The scripts in package.json 4
2.2 The dependencies in package.json 5
2.3 .npmrc 5
2.4 tsconfig.json 5
2.5 vite.config.ts 6
2.6 index.html 6
2.7 metadata.json 6
3. Environment and ignore files 6
3.1 .env.example 6
3.2 .gitignore and .dockerignore 6
4. Deployment files 6
4.1 vercel.json 6
4.2 Dockerfile 7

4.3 deploy-azure.sh....7
4.4 azuredeploy.json....7
4.5 azure-pipelines.yml....7
5. Database files....7
5.1 supabase/schema.sql, the recommended route....7
5.2 prisma/schema.prisma and prisma/migrations/0_init/migration.sql....8
5.3 prisma/migrations/migration_lock.toml....8
5.4 prisma/seed.ts....8
6. Scripts....8
6.1 scripts/localdb.mjs....8
6.2 scripts/persistence-test.mjs....8
6.3 scripts/seed-supabase.js....9
6.4 scripts/seed-supabase-workbooks.ts....9
7. Content and documents....9
8. The build pipeline at a glance....9

1. Introduction

This guide covers every file that sits outside `server/` and `src/`: the project settings, the build, the deployment files, the database files and the helper scripts. It explains what each file does and why it is written the way it is.

Please read it alongside the Deployment Guide, which gives the step-by-step procedures. The application code itself is explained in Code Guide: Backend and Code Guide: Frontend. Everything here matches commit `0cc8c36`.

Group	Files	Where it is covered
Project settings	package.json, .npmrc, tsconfig.json, vite.config.ts, index.html and metadata.json	Section 2
Environment and ignore files	.env.example, .gitignore and .dockerignore	Section 3
Deployment	vercel.json, Dockerfile, deploy-azure.sh, azuredeploy.json and azure-pipelines.yml	Section 4
Database	supabase/schema.sql, prisma/schema.prisma, prisma/migrations and prisma/seed.ts	Section 5
Scripts	scripts/localdb.mjs, scripts/persistence-test.mjs, scripts/seed-supabase.js and scripts/seed-supabase-workbooks.ts	Section 6
Content and documents	uploads/, public/, README.md, PROJECT_FLOW.md and docs/	Section 7

2. Project settings

2.1 The scripts in package.json

Script	What it runs	What it is for
`dev`	`tsx server.ts`	The development server on port 3000, with the API and the frontend together
`start`	`tsx server.ts`	The production server for Docker and Azure. It serves `dist/` when `NODE_ENV=production`
`build`	`vite build && esbuild server/vercel.ts --bundle --platform=node --target=node22 --format=esm --outfile=api/index.js --packages=external`	Builds the frontend into `dist/`, and bundles the API into a single file for Vercel
`lint`	`tsc --noEmit`	A type check only. The project has no ESLint
`db:start`, `db:stop` and `db:status`	`node scripts/localdb.mjs` with `start`, `stop` or `status`	Control a local PostgreSQL on port 5433
`db:push` and `db:deploy`	The Prisma command-line tool	Create the tables from the Prisma schema. This is optional, and `supabase/schema.sql` is the recommended route
`db:seed`	`tsx prisma/seed.ts`	Only seeds the in-memory store, as section 5.4 explains

Script	What it runs	What it is for
`test:persistence`	A build, then `scripts/persistence-test.mjs`	The end-to-end persistence test
`preview` and `clean`	The Vite preview, and removing `dist`	Rarely needed

2.2 The dependencies in package.json

Package	Why it is there
express, cors and multer	The web server, requests from other origins, and file uploads
exceljs	Reading workbooks along with their styles, and writing the MR11 export
pg	Sending SQL to Supabase
jsonwebtoken and bcryptjs	Session tokens, and hashing the passwords of accounts created in Admin Permissions
react, react-dom, react-router-dom, zustand and axios	The frontend
chart.js, react-chartjs-2 and lucide-react	Charts and icons
vite, @vitejs/plugin-react, @tailwindcss/vite and tailwindcss	Building and styling the frontend
tsx, esbuild and typescript	Running and bundling TypeScript
@prisma/client and prisma	Installed, but the client import is pointed at the in-memory store (see section 2.4), and the command-line tool is only needed for the optional `db:push`
@google/genai, zod and motion	The Gemini SDK and libraries for validation and animation. The code uses none of them, so they can be removed
dotenv	Loads `.env` when the server starts

2.3 .npmrc

```
legacy-peer-deps=true
```

Vite 8 and the version of esbuild that tsx expects disagree about a peer dependency, and by default npm refuses to install when that happens. This line makes every install accept it, whether it runs locally, in Docker, on Azure or on Vercel. Please keep it.

2.4 tsconfig.json

Setting	What it does
`target ES2022`, `module ESNext` and `moduleResolution bundler`	Produce modern JavaScript, and find imports the same way Vite and esbuild do
`jsx react-jsx`	Use React's automatic JSX runtime
`noEmit`, `allowImportingTsExtensions` and `isolatedModules`	TypeScript only checks the code, while Vite, tsx and esbuild do the compiling

Setting	What it does
`paths "@/*"`	Makes `@/` mean the top of the project
`paths "@prisma/client"`, set to `./server/db/prisma.ts`	Makes every `import { PrismaClient } from '@prisma/client'` load the in-memory store. Both tsx and esbuild follow this setting

2.5 vite.config.ts

This adds the React and Tailwind plugins and the `@` alias. When `DISABLE_HMR=true` is set, it switches off live reloading and file watching, which helps in hosted editors.

2.6 index.html

This is the application's only HTML page. It holds the title, the favicon and the `<div id="root">` element that React draws into, and it loads `src/main.tsx`.

2.7 metadata.json

A description file used by Google AI Studio, where the project may first have been started. It declares a server-side Gemini feature that the code doesn't use, and it has no effect on Vercel, Azure or local runs.

3. Environment and ignore files

3.1 .env.example

This is a template for `.env`. It lists `APP_URL` (used only by AI Studio and never read by the code), `JWT_SECRET`, `DATABASE_URL`, the Azure ids and `USD_TO_MYR_RATE`. Copy it to `.env` and fill in the real values, and never commit `.env` itself. At the moment the file holds the real Supabase project reference and Azure subscription id, which should be replaced with placeholders.

3.2 .gitignore and .dockerignore

`.gitignore` keeps `node_modules`, `dist`, log files, `.env*` (apart from the example), `package-lock.json`, the local database folder `.localdb/` and `.vercel` out of Git. `.dockerignore` keeps `node_modules`, `dist`, `.git` and `.env*` out of the Docker build.

Because `package-lock.json` is ignored, installs aren't tied to exact package versions. We suggest committing a lock file.

4. Deployment files

4.1 vercel.json

```
"buildCommand": "npm run build",
"outputDirectory": "dist",
"regions": ["sin1"],
"rewrites": [
  { "source": "/api/(.*)", "destination": "/api/index" },
  { "source": "/health", "destination": "/api/index" },
  { "source": "/(.*)", "destination": "/index.html" }
```

```
]
```

Vercel builds the project and serves `dist/` as static files. It runs `api/index.js` as a serverless function in Singapore for `/api/*` and `/health`, and sends every other address to `index.html`, so that the app's own addresses still work when a page is refreshed.

4.2 Dockerfile

The image is built in two stages on `node:22-alpine`:

1. **The builder stage** copies `package.json` and `.npmrc`, runs `npm install`, copies the source code, and then runs `npm run lint` and `npm run build`.
2. **The runner stage** sets `NODE_ENV=production` and `PORT=3000`, installs only the production dependencies, and copies `dist/`, `server/`, `server.ts`, `tsconfig.json` and `public/`. It then switches to the `node` user, which isn't root, and runs `npm start`.

Because `npm start` runs `tsx server.ts`, the running image still needs `tsx`, and it needs `tsconfig.json` for the `@prisma/client` mapping. Both are included.

4.3 deploy-azure.sh

This is a one-off script, and `DATABASE_URL` must be set before you run it. It goes through these steps:

1. It signs in to the Azure tenant and selects the subscription.
2. It creates the resource group `rg-mfe-formwork-prod` (in `eastus` unless you say otherwise), a Linux B1 plan, and a Node 22 web app with a random name.
3. It sets the app settings, including a random `JWT_SECRET`.
4. It builds the project, zips it up without `node_modules` and `.git`, and deploys the zip.

Change `LOCATION` and the tenant and subscription ids before you use it.

4.4 azuredeploy.json

This is an Azure Resource Manager template. Its parameters are `webAppName`, `location`, `sku`, `databaseUrl`, `jwtSecret`, `repoUrl` and `branch`. It creates an App Service plan, a web app, and a source-control link that deploys from the Git repository and branch you give it.

4.5 azure-pipelines.yml

This is an Azure DevOps pipeline with two stages. The first runs on `main` and `azure-deploy`: it installs the packages, runs `npm run lint` and `npm run build`, and zips and publishes the result. The second deploys the zip to the web app, but only from `azure-deploy`.

The pipeline writes `JWT_SECRET` in plain text in `appSettings`. Move it to a secret variable before using the pipeline, and add `DATABASE_URL` as well.

5. Database files

5.1 supabase/schema.sql, the recommended route

Run this once in the Supabase SQL editor. It creates the three enum types (written so that running them again doesn't fail), the eleven tables using `IF NOT EXISTS`, the unique indexes, and these starting rows:

Table	Rows it adds
Role	The nine roles, with fixed ids from `role-admin` to `role-dispatch`
Department	The seven departments, from `dept-bd` to `dept-dispatch`
User	The main admin account, whose password is stored as the plain text `admin123` (which the sign-in accepts), and eight demonstration accounts stored as PBKDF2 hashes of `admin123`
UserRole	The role for each account
Mr11Config	The `singleton` row

Every insert uses `ON CONFLICT` with `DO NOTHING`, so running the file a second time changes nothing. The file declares no foreign keys.

5.2 prisma/schema.prisma and prisma/migrations/0_init/migration.sql

These describe the same tables for Prisma, with a matching SQL migration. The layout differs in places, and unlike the Supabase file it declares foreign keys. For example, Mr11Run keeps its rows in `records`, and FileVersion has no `storagePath`. The data dictionary appendix of the System Architecture Document lists every difference.

`binaryTargets` lists the platforms that Prisma's engine would need on Vercel, on Azure and locally. Since the application doesn't use the Prisma client, this file only matters if you run `npm run db:push`.

5.3 prisma/migrations/migration_lock.toml

This records that the migrations are written for PostgreSQL.

5.4 prisma/seed.ts

This seeds the roles, departments and users through `PrismaClient`. Because that name points at the in-memory store, the seed fills memory and then exits without writing anything to Supabase. Use `supabase/schema.sql` instead.

6. Scripts

6.1 scripts/localdb.mjs

This starts, stops or reports on a local PostgreSQL on port 5433, keeping its data in `.localdb/data` and using `pg_ctl`. Set `PG_BIN` if PostgreSQL isn't installed in `C:/Program Files/PostgreSQL/18/bin`. Before the first start, create the data folder once with `initdb`.

6.2 scripts/persistence-test.mjs

This is an end-to-end test of what matters most on serverless hosting: that data survives a restart, and that several server instances agree with each other. It needs a throwaway local database in `TEST_DATABASE_URL`, which must be on localhost because the test wipes it, and a folder of department workbooks. The test works through five steps:

1. It creates the database again from `supabase/schema.sql`.
2. It starts the built server (`api/index.js`, the same file Vercel runs), signs in, and uploads every workbook through the API.

3. It creates, changes and deletes users, and uploads a newer BD file.
4. It restarts the server and checks that the uploads, users, passwords, series history and MR11 all survived.
5. It runs two servers side by side and checks that each sees the users and uploads made on the other, and that an MR11 built on one uses the newer workbook uploaded to the other.

It prints PASS or FAIL for each of its 23 checks. Two examples are "first request after restart already shows bd-v2.xlsx" and "upload on server A builds MR11 from design-v2, not the older design".

```
TEST_DATABASE_URL="postgres://postgres@127.0.0.1:5544/mfe_test?sslmode=disable" \
  node scripts/persistence-test.mjs <folder with bd.xlsx, finance.xlsx, ... dispatch.xlsx>
```

6.3 scripts/seed-supabase.js

This adds the eight demonstration accounts and their roles to the database named in `DATABASE_URL`. Only use it on a test database, because production shouldn't have demonstration accounts.

6.4 scripts/seed-supabase-workbooks.ts

This reads the newest sample workbook for each department in `uploads/`, using the same Excel reader as the application, and saves each one as that department's active FileVersion in the database. It is handy for filling a staging database, but please don't run it against production.

7. Content and documents

Path	What it is
`uploads/`	16 sample department workbooks, which are loaded when there is no database. Remove them from the repository if they contain real client data
`public/`	The logo (`doka mfe logo.png.png`), the favicon and the icons, served exactly as they are
`README.md`	A short overview with the commands for running the project. Its line about the default password is out of date
`PROJECT_FLOW.md` and `docs/*.md`	Early design and deployment notes, some of which describe features that were never built. This documentation set replaces them

8. The build pipeline at a glance

Step	Tool	What goes in	What comes out
Type check	`tsc --noEmit`	Every `.ts` and `.tsx` file	Errors only
Frontend build	Vite, with React and Tailwind	`index.html` and `src/`	`dist/index.html`, plus the `.js` and `.css` files in `dist/assets/`
API bundle	esbuild	`server/vercel.ts` and everything it imports	`api/index.js`, with the packages left outside the bundle
Running locally	tsx	`server.ts`	A server on port 3000, with Vite's development middleware

Step	Tool	What goes in	What comes out
Running in production	tsx, or Vercel	`server.ts` and `dist/`, or `api/index.js` and `dist/`	The live application